

Universidad Central del Ecuador

Dispositivos Móviles

Grupo 4

Documentación – Lógica de la aplicación

Aplicación con Base de Datos Local y en la Nube

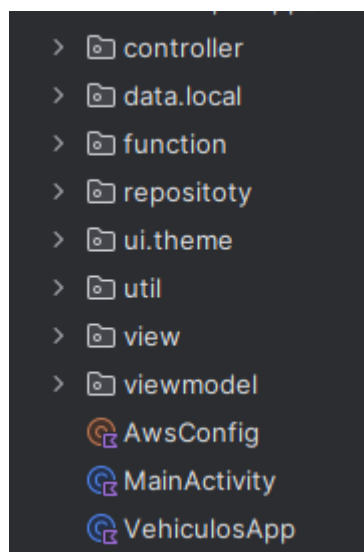
1. Arquitectura: MVC

Nuestra aplicación hace uso del patrón de arquitectura MVC, lo que hace que cuente con 3 paquetes principales:

- Controller
- Model
- View

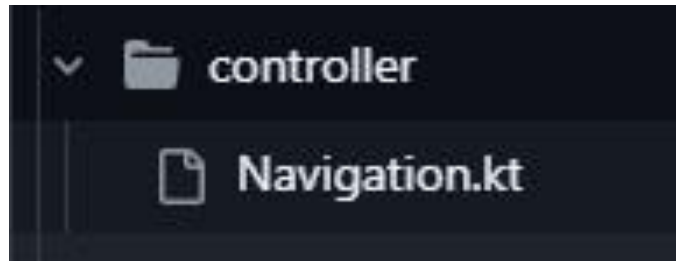
Adicionalmente, cuenta con el paquete Ui/theme (para los estilos). Dentro de todos estos paquetes, se encuentran alojadas las clases que contienen el código fuente.

Y por último una capa de persistencia almacenada en data.local y repository mediante SQLite y Dynamo DB



2. Paquetes y clases

Controller



- **Navigation.kt**

Es un composable que define la estructura de navegación de la aplicación utilizando NavHost de Jetpack Compose. Gestiona la navegación entre diferentes pantallas (splash screen, login, registro, pantalla principal, añadir vehículo y editar vehículo) y mantiene el estado de las listas de usuarios y vehículos, así como un vehículo seleccionado para edición. Utiliza conceptos de Jetpack Compose como remember, mutableStateListOf, y LaunchedEffect para manejar el estado y efectos secundarios.

Flujo de Navegación

1. La aplicación inicia en la pantalla de splash, espera 2 segundos y navega al login.
2. Desde el login, el usuario puede:
 - Iniciar sesión y navegar al home.
 - Ir a la pantalla de registro para crear un nuevo usuario.
3. Desde el home, el usuario puede:
 - Cerrar sesión y volver al login.
 - Navegar a addVehiculo para añadir un vehículo.
 - Seleccionar un vehículo para editarlo y navegar a editVehiculo.
 - Eliminar un vehículo directamente.
4. Desde addVehiculo o editVehiculo, el usuario puede guardar los cambios o cancelar, regresando al home.

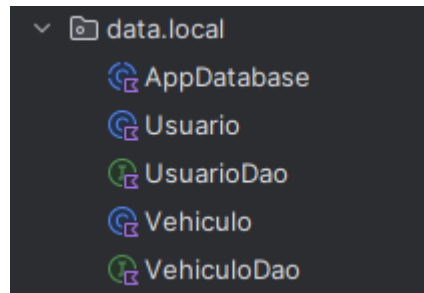
Composables y Estado

- Uso de remember: Asegura que las listas de usuarios y vehículos, así como el vehículo seleccionado, persistan durante las recomposiciones de la UI.
- Uso de mutableStateListOf: Permite que las listas sean reactivas, es decir, los cambios en las listas (añadir, eliminar, actualizar) provocan que la UI se actualice automáticamente.
- Uso de LaunchedEffect: Ejecuta el retraso y la navegación en la pantalla de splash de forma controlada, evitando múltiples ejecuciones innecesarias.

- Navegación: Utiliza `navController.navigate` para mover entre pantallas y `popBackStack` para regresar. La opción `popUpTo` en la pantalla de splash asegura que no se pueda volver a ella.

Data.local

- Este viene hacer nuestra nueva capa de model que contiene los modelos de Usuario y Vehiculo,



• AppDatabase

- La clase `AppDatabase` define la base de datos de la aplicación utilizando la librería `Room`, incluyendo las entidades `Usuario` y `Vehiculo`. Proporciona un patrón `Singleton` para garantizar una única instancia de la base de datos en toda la app. Además, implementa un `RoomDatabase.Callback` que permite la precarga de datos iniciales en la tabla de usuarios (con cuentas administradoras) y vehículos al momento de la creación de la base de datos. Esta configuración facilita una gestión eficiente y segura de los datos, integrándose con el modelo de arquitectura de Android para soportar operaciones asincrónicas mediante corrutinas.

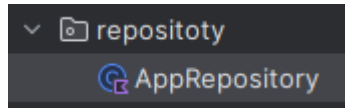
• UsuarioDao

- La interfaz `UsuarioDao` define las operaciones de acceso a datos para la entidad `Usuario` utilizando `Room` en Android. Incluye métodos para insertar o reemplazar usuarios en la base de datos (`insertarUsuario`), consultar un usuario específico por su nombre (`getUsuarioPorNombre`), y obtener un flujo reactivo con la lista de todos los usuarios (`getAllUsuarios`). Estas operaciones permiten una gestión eficiente y actualizada de los datos, integrándose fácilmente con arquitecturas basadas en corrutinas y programación reactiva.

• VehiculoDao

- La interfaz `VehiculoDao` define las operaciones de acceso a datos para la entidad `Vehiculo` en una base de datos gestionada por `Room` en Android. Ofrece métodos para insertar o reemplazar un vehículo (`insertarVehiculo`), actualizar registros existentes (`actualizarVehiculo`), eliminar vehículos (`eliminarVehiculo`), y obtener un flujo reactivo de la lista completa de vehículos ordenada alfabéticamente por marca (`getAllVehiculos`). Estas funciones permiten realizar operaciones CRUD de manera eficiente y mantener sincronizados los datos con la interfaz de usuario mediante programación reactiva.

Repository



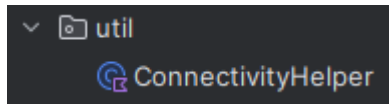
• AppRepository

- La clase AppRepository actúa como un intermediario entre la base de datos local (Room) y la base de datos en la nube (AWS DynamoDB). Ofrece un conjunto de métodos para gestionar vehículos y usuarios, permitiendo operaciones CRUD tanto en local como en la nube. Implementa un mecanismo de sincronización bidireccional que sube datos locales a DynamoDB y descarga datos de la nube a la base local, asegurando que ambas estén actualizadas. La clase utiliza un ConnectivityHelper para verificar la disponibilidad de red antes de realizar sincronizaciones. Además, expone flujos reactivos de los datos locales y proporciona funciones para validar contraseñas seguras. De esta forma, AppRepository centraliza la lógica de acceso y sincronización de datos, garantizando consistencia y eficiencia en la gestión de la información de la aplicación.
- **Uso de corutinas**

```
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.withContext
```

- Permiten gestionar operaciones asíncronas de manera eficiente y segura. Dispatchers se usa para especificar en qué hilo o contexto de ejecución se deben correr las corutinas; por ejemplo, Dispatchers.IO se emplea para operaciones de entrada/salida como acceso a base de datos o red. Flow representa un flujo de datos reactivo que emite actualizaciones de manera asíncrona, ideal para observar cambios en la base de datos. La función first() permite obtener de manera suspendida el primer valor emitido por un Flow, útil en procesos de sincronización. Finalmente, withContext permite cambiar de contexto de ejecución dentro de una corutina, garantizando que operaciones que puedan bloquear (como acceso a la red o a la base de datos) no afecten el hilo principal de la aplicación.

Util



- **ConnectivityHelper**

- Encapsula la lógica para verificar la disponibilidad de conexión a internet en la aplicación. A través de su método `isNetworkAvailable()`, que requiere el permiso `ACCESS_NETWORK_STATE`, consulta el servicio del sistema `ConnectivityManager` para obtener el estado de la red activa. Luego, mediante `NetworkCapabilities`, determina si la conexión disponible es a través de WiFi, red celular o Ethernet. Si alguna de estas conexiones está activa, devuelve `true`, indicando que hay acceso a internet; en caso contrario, devuelve `false`. Esta clase permite que otras partes del código verifiquen de forma sencilla si es posible realizar operaciones en la nube o sincronizaciones.

Objetos Para Realizar y configurar la Conexión



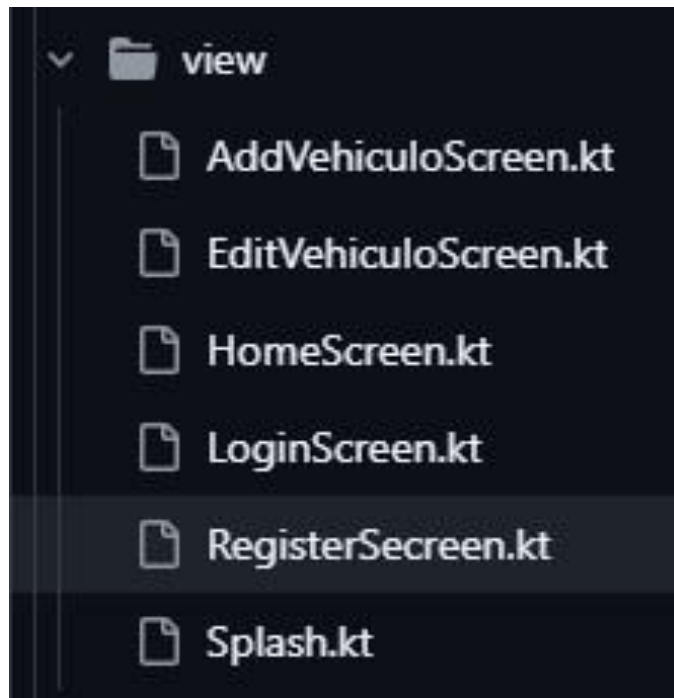
- **AwsConfig**

- `AwsConfig` actúa como una configuración centralizada para inicializar y proporcionar un cliente de `AWS DynamoDB`. Define tres constantes: `ACCESS_KEY`, `SECRET_KEY` y `SESSION_TOKEN`, que representan las credenciales temporales necesarias para autenticar la conexión con `AWS`. También especifica la región de `AWS` que se usará (`US_EAST_1`). El cliente de `DynamoDB` (`dynamoDbClient`) se inicializa de forma perezosa (`lazy`), es decir, solo se crea cuando se usa por primera vez, utilizando dichas credenciales y configuraciones. Este cliente permite realizar operaciones de lectura, escritura y sincronización de datos entre la app y la base de datos en la nube de `AWS DynamoDB`. En resumen, `AwsConfig` facilita el acceso seguro y consistente a los servicios de `AWS` desde cualquier parte de la app.

- **VehiculosApp**

- Actúa como una clase de configuración global para la aplicación. Al ser instanciada al inicio del ciclo de vida de la app, crea de forma perezosa (`lazy`) una instancia de la base de datos `AppDatabase` usando el contexto de la aplicación, garantizando que la base de datos sea un singleton en toda la app. Además, también instancia de manera perezosa un objeto `AppRepository`, que centraliza la lógica de acceso a datos y sincronización con la nube, utilizando los DAOs de `Usuario` y `Vehiculo` proporcionados por la base de datos. De esta forma, `VehiculosApp` permite que cualquier parte de la app acceda fácilmente a un único repositorio de datos consistente y correctamente configurado.

View



- **AddVehiculoScreen.kt**

Es un composable que proporciona una interfaz para añadir o editar un vehículo en la aplicación. Se utiliza en el flujo de navegación para crear un nuevo vehículo o modificar uno existente.

Parámetros:

- **onSave: (Vehiculo) -> Unit:** Callback para guardar el vehículo creado/editado.
- **onCancel: () -> Unit:** Callback para cancelar la operación y regresar.
- **vehiculoAEditar: Vehiculo? = null:** Vehículo opcional para prellenar los campos en modo edición.

Estado:

- **Propiedades Mutables:** Usa `mutableStateOf` para almacenar `placa`, `marca`, `anio`, `color`, `costoPorDia`, `activo`, `imagenUri` y `showErrors`. Si `vehiculoAEditar` no es nulo, inicializa los campos con sus valores.
- **Validaciones:** Verifica si los campos son válidos (`isPlacaValid`, `isMarcaValid`, etc.) para mostrar errores si es necesario.
- **Selección de Imagen:** Usa `rememberLauncherForActivityResult` con `ActivityResultContracts.GetContent` para seleccionar una imagen desde la galería (`imagenUri`).

Interfaz:

- **Diseño:** Un Column con desplazamiento vertical (verticalScroll) que contiene:
 - o Un título ("Nuevo Vehículo" o "Editar Vehículo").

○ Campos de texto (OutlinedTextField) para placa, marca, año, color y costoPorDia, con validaciones y mensajes de error. ○ Un Checkbox para activo. ○ Un botón para seleccionar una imagen desde la galería, que se muestra con Image si se selecciona.

- Botones "Guardar" y "Cancelar" en un Row.

- **Comportamiento:**

- **Guardar:** Valida los campos; si son válidos, crea un objeto Vehiculo y llama a onSave. Si no hay imagen seleccionada y no se está editando, usa R.drawable.toyota como imagenResId.
- **Cancelar:** Llama a onCancel para regresar sin guardar.
- **Errores:** Muestra mensajes de error si los campos no son válidos al intentar guardar (showErrors = true).

• [EditVehiculoScreen.kt](#)

Es un composable de que proporciona una interfaz para editar un vehículo existente en la aplicación.

Parámetros:

- **vehiculo: Vehiculo:** El vehículo a editar, usado para prellenar los campos.
- **onSave: (Vehiculo) -> Unit:** Callback para guardar el vehículo editado.
- **onCancel: () -> Unit:** Callback para cancelar y regresar.

Estado:

- **Propiedades Mutables:** Usa mutableStateOf para placa, marca, año, color, costoPorDia, activo, imagenUri, imagenSeleccionada (para imágenes precargadas) y expanded (para el menú desplegable). Inicializa los campos con los valores del vehiculo.
- **Selección de Imagen:** Usa rememberLauncherForActivityResult para seleccionar una imagen de la galería (imagenUri). También permite elegir imágenes precargadas (toyota, chevrolet, nissan) si no hay imagenUri.
- **Imágenes Precargadas:** Un mapa (imagenes) asocia nombres con recursos (R.drawable.toyota, etc.), y imagenSeleccionada rastrea la selección.

Interfaz:

- **Diseño:** Un Column con desplazamiento vertical (verticalScroll) que incluye:
 - Título "Editar Vehículo".

-
- Campos de texto (OutlinedTextField) para placa, marca, año, color y costoPorDia. ○ Un Checkbox para activo. ○ Un botón para seleccionar una imagen de la galería.
- Un ExposedDropDownMenuBox para elegir imágenes precargadas (solo si no hay imagenUri).
- Una vista previa de la imagen seleccionada (Image) usando rememberAsyncImagePainter para imagenUri o painterResource para imágenes precargadas.
- Botones "Guardar" y "Cancelar" en un Row.

- Comportamiento:

- **Guardar:** Crea un nuevo objeto Vehiculo con los datos editados y llama a onSave. Usa la imagen precargada si no hay imagenUri, con R.drawable.toyota como predeterminado.
- **Cancelar:** Llama a onCancel para regresar sin guardar.

• HomeScreen.kt

Es un composable que muestra la pantalla principal de la aplicación, presentando una lista de vehículos y opciones para agregar un vehículo nuevo, editar uno existente o cerrar sesión.

Parámetros:

- **vehiculos: List<Vehiculo>:** Lista de vehículos a mostrar.
- **onLogout: () -> Unit:** Callback para cerrar sesión.
- **onAddVehiculo: () -> Unit:** Callback para navegar a la pantalla de añadir vehículo.
- **onEditVehiculo: (Vehiculo) -> Unit:** Callback para editar un vehículo.
- **onDeleteVehiculo: (Vehiculo) -> Unit:** Callback para eliminar un vehículo.
- **modifier: Modifier = Modifier:** Modificador opcional para personalizar el diseño.

Interfaz:

- **Diseño:** Un Column con:
 - Un Row con un título ("Inicio"), un botón para añadir vehículo y otro para cerrar sesión.
 - Un LazyColumn que muestra una lista de vehículos en tarjetas (Card).
- **Tarjetas de Vehículo:**
 - Cada tarjeta muestra una imagen (de imagenUri con rememberAsyncImagePainter, imagenResId con painterResource, o R.drawable.toyota por defecto).

-
- Muestra detalles: placa, marca, año, color, costoPorDia y activo.
- Incluye botones "Editar" y "Eliminar" en un Row.

- **Comportamiento:**

- **Agregar:** Llama a onAddVehiculo para navegar a la pantalla de añadir vehículo.
- **Cerrar Sesión:** Llama a onLogout para volver al login.
- Editar:** Llama a onEditVehiculo con el vehículo seleccionado.
- **Eliminar:** Llama a onDeleteVehiculo para eliminar el vehículo de la lista.

• **LoginScreen.kt**

Es un composable que proporciona una interfaz para que los usuarios inicien sesión en la aplicación verificando sus credenciales (nombre y apellido) contra una lista de usuarios registrados.

Parámetros:

- **usuariosRegistrados: List<Usuario>:** Lista de usuarios registrados para validar las credenciales.
- **onLoginSuccess: () -> Unit:** Callback para navegar a la pantalla principal si el login es exitoso.
- **onGoToRegister: () -> Unit:** Callback para navegar a la pantalla de registro.
- **modifier: Modifier = Modifier:** Modificador opcional para personalizar el diseño.

Estado:

- **Propiedades Mutables:** Usa mutableStateOf para nombre y apellido (como TextFieldValue) y errorMsg (para mensajes de error).
- **Validación:** Busca un usuario en usuariosRegistrados comparando nombre y apellido (sin distinguir mayúsculas/minúsculas).

Interfaz:

- **Diseño:** Un Column centrado con:
 - Título "Login".
 - Campos de texto (OutlinedTextField) para nombre y apellido.
 - Botón "Ingresar" para validar credenciales.
 - Mensaje de error si las credenciales son incorrectas.
 - Botón de texto ("¿No tienes cuenta? Regístrate") para navegar al registro.
- **Comportamiento:**

-
- **Ingresar:** Verifica si existe un usuario con el nombre y apellido ingresados. Si coincide, limpia errorMsg y llama a onLoginSuccess. Si no, muestra un mensaje de error.
- **Registro:** Llama a onGoToRegister para navegar a la pantalla de registro.

- RegisterScreen.kt

Es un composable que proporciona una interfaz para registrar un nuevo usuario en la aplicación, validando que los datos no estén vacíos y que el usuario no exista previamente.

Parámetros:

- **usuarios: List<Usuario>:** Lista de usuarios registrados para verificar duplicados.
- **onRegisterSuccess: (Usuario) -> Unit:** Callback para guardar el nuevo usuario y navegar hacia atrás.
- **modifier: Modifier = Modifier:** Modificador opcional para personalizar el diseño.

Estado:

- **Propiedades Mutables:** Usa mutableStateOf para nombre y apellido (como TextFieldValue) y errorMsg (para mensajes de error).
- **Validaciones:** Comprueba que nombre y apellido no estén vacíos y que no exista un usuario con los mismos datos (ignorando mayúsculas/minúsculas).

Interfaz:

- **Diseño:** Un Column centrado con:
 - o Título "Registro".
 - o Campos de texto (OutlinedTextField) para nombre y apellido.
 - o Botón "Registrarse" para procesar el registro.
 - o Mensaje de error si los campos están vacíos o el usuario ya existe.
- **Comportamiento:**
 - o **Registrarse:** Valida los campos. Si son válidos y el usuario no está registrado, crea un Usuario y llama a onRegisterSuccess. Si no, muestra un mensaje de error.

• Splash.kt

Es un composable que muestra una pantalla de bienvenida con un logo y un texto animado al iniciar la aplicación.

Estado:

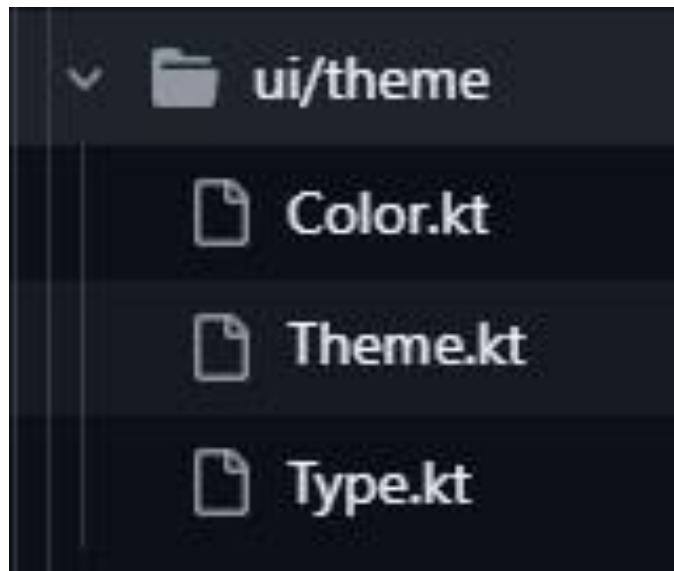
- **Propiedad Mutable:** Usa mutableStateOf para visible (inicialmente false), que controla la animación.
- **Efecto:** Usa LaunchedEffect para establecer visible en true al cargar el composable, activando la animación.

Interfaz:

- **Diseño:** Un Box centrado que contiene un AnimatedVisibility con:
 - o Una animación de entrada (fadeIn y scaleIn, duración de 1000 ms, escala inicial de 0.7).
 - o Un Column con:

- ▣ Una imagen (Image) del logo (R.drawable.logo2), tamaño 220 dp.
 - ▣ Un Spacer de 32 dp.
 - ▣ Un texto ("Vehiculos Don Veyker") con estilo personalizado (fuente Pacifico, tamaño 36 sp, color azul).
- **Comportamiento:** Muestra el logo y el texto con una animación de fundido y escalado al cargar.

Ui/theme



Las tres clases trabajan juntas para definir el tema visual de la aplicación:

- **Color.kt** establece una paleta de colores para temas claro y oscuro.
- **Theme.kt** implementa el tema (App_VehiculosTheme), seleccionando esquemas de color (estáticos o dinámicos) y aplicando tipografía.
- **Type.kt** define estilos tipográficos, asegurando consistencia en los textos.

App_VehiculosTheme combina colores y tipografía, aplicándolos a toda la aplicación mediante MaterialTheme. Soporta modo claro/oscuro y colores dinámicos.

3. MainActivity.kt

Es la actividad principal de la aplicación Android App_Vehiculos, que configura la interfaz de usuario utilizando Jetpack Compose y gestiona la navegación entre pantallas de manera alternativa a AppNavigation.

Estructura:

- **Clase MainActivity:**

- **Propósito:** Extiende ComponentActivity y configura el contenido de la aplicación. ◦

- Método onCreate:**

- ▢ Llama a setContent para envolver la aplicación en App_VehiculosTheme (aplicando el tema definido en ui.theme) y renderiza AppNavigation como el composable principal.

- **Uso:** Actúa como punto de entrada de la aplicación, inicializando el tema y la navegación.

- **Composable App:**

- **Propósito:** Gestiona la navegación entre pantallas (login, register, home, addVehiculo) utilizando un estado (currentScreen) en lugar de NavHost.

- **Estado:**

- ▢ **currentScreen:** Controla la pantalla actual (inicialmente "login").
 - ▢ **usuariosRegistrados:** Lista mutable de Usuario con valores iniciales para autenticación/registro.
 - ▢ **vehiculos:** Lista mutable de Vehiculo con datos iniciales.
 - ▢ **vehiculoEnEdicion:** Almacena el vehículo en edición (o null para añadir uno nuevo).

- **Interfaz:**

- ▢ Usa un bloque when para renderizar una pantalla según currentScreen:
 - ▢ **login:** LoginScreen verifica credenciales y permite ir a registro.
 - ▢ **register:** RegisterScreen añade nuevos usuarios a usuariosRegistrados y vuelve a login.
 - ▢ **home:** HomeScreen muestra vehículos, permite cerrar sesión, añadir, editar o eliminar vehículos.
 - ▢ **addVehiculo:** AddVehiculoScreen crea o edita un vehículo, actualizando vehiculos y regresando a home.

- **Comportamiento:**

- ▢ **Navegación:** Cambia currentScreen para alternar pantallas.

- ▢ **Gestión de Datos:** Actualiza usuariosRegistrados y vehiculos según las acciones (registro, añadir/editar/eliminar vehículos).
- ▢ **Edición:** Usa vehiculoEnEdicion para pasar un vehículo a AddVehiculoScreen en modo edición.

4. Archivos de Configuración

- Manifest

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

El primer permiso permite que la app pueda usar la conexión a Internet para enviar o recibir datos, por ejemplo, para conectarse a servidores, descargar contenido, usar APIs web, etc.

El segundo permite que la app revise si hay conexión a Internet o a alguna red (WiFi, datos móviles) y cuál es su estado (conectado, desconectado, tipo de red). Es útil para saber cuándo intentar o no hacer operaciones en línea.

- Gradle.kts

Implementación de las librerías de persistencia y base de datos que vamos a usar, SQLite y Dynamo DB

```
// --- Room (Base de datos SQLite) ---
implementation(libs.room.runtime)
implementation(libs.room.ktx)
ksp(libs.room.compiler) // KSP procesará las anotaciones de Room

// --- AWS SDK para DynamoDB ---
// El BOM (Bill of Materials) nos ayuda a gestionar las versiones de las librerías de AWS
implementation(platform("software.amazon.awssdk:bom:2.20.26"))
implementation("software.amazon.awssdk:dynamodb")
implementation("software.amazon.awssdk:url-connection-client")
```

Diagrama de Flujo – Sincronización SQLite y DynamoDB

Diagrama de Flujo: Sincronización SQLite y DynamoDB

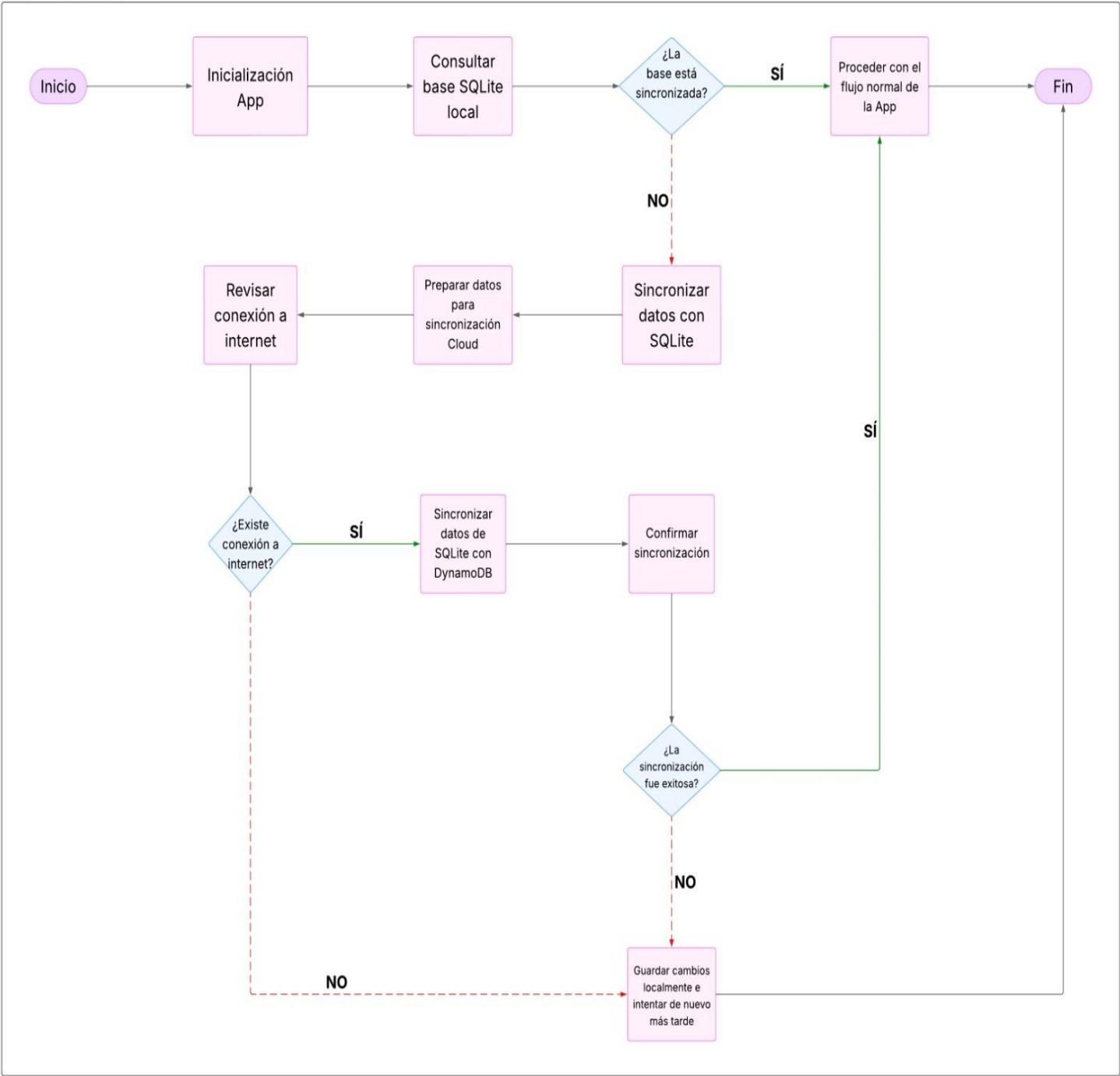


Tabla de Usuarios de la vista de Android

usuariosvehiculos

Live updates

	nombre	passwordHash	esAdmin
1	Byron	c57a183476266022072cc94fbde250daceda42c37bfe71a06c44bfc1cb02f004	1
2	Angelo	7835b5b3cf71b703d247aa67b93a6be371c8bca826fe193379e51ff3c8df020f	1
3	Kevin	0169f3dad52f164528ceba132477d35ce45e7987c9a032accffa3c002d01853	1
4	Joffre	531b34617e2da6ed23dcfe85e1feeafc3179dbb56fff1df620905907f74d1b9d	1
5	Cristian	cba1bc17ed3fe28041ee6a80094fa1023b6cfc77bd5b0adbbd49608cf877a22d	1
6	Jordi	d18314cb02ec690206d6445bf6770595a18f3ab3ade497ae8594d2703f01b2d1	1
7	Edgar	c26447b5df3186ae36fb403c3b35c05d8484e593c822ac097251974a45d0ad54	1
8	Michael	f1d40ef3b6da78bc7f6be3f38c909b540aac701d7e8d104a9ae06ecb664b08f4	1

Tabla de Usuarios de aws

Table: Usuarios - Items returned (8)

Scan started on June 19, 2025, 23:18:33

	nombre (String)	esAdmin	passwordHash
☐	Byron	true	c57a183476266022072cc94fbde250daceda42c37bfe71a06c44bfc1cb02f004
☐	Angelo	true	7835b5b3cf71b703d247aa67b93a6be371c8bca826fe193379e51ff3c8df020f
☐	Kevin	true	0169f3dad52f164528ceba132477d35ce45e7987c9a032accffa3c002d01853
☐	Joffre	true	531b34617e2da6ed23dcfe85e1feeafc3179dbb56fff1df620905907f74d1b9d
☐	Cristian	true	cba1bc17ed3fe28041ee6a80094fa1023b6cfc77bd5b0adbbd49608cf877a22d
☐	Jordi	true	d18314cb02ec690206d6445bf6770595a18f3ab3ade497ae8594d2703f01b2d1
☐	Edgar	true	c26447b5df3186ae36fb403c3b35c05d8484e593c822ac097251974a45d0ad54
☐	Michael	true	f1d40ef3b6da78bc7f6be3f38c909b540aac701d7e8d104a9ae06ecb664b08f4

Tabla de Vehículos de la vista de Android

usuariosvehiculos

Live updates

	placa	marca	año	color	costoPorDia
1	ABC123	Toyota	2020	Rojo	50.0
2	TIL777	Mazda	2018	Rojo	35.0
3	AUC455	Hyundai	2025	Negro	75.0
4	DEF456	Nissan	2022	Azul	60.0
5	XYZ789	Chevrolet	2019	Negro	45.0

Tabla de Vehículos de la vista aws.

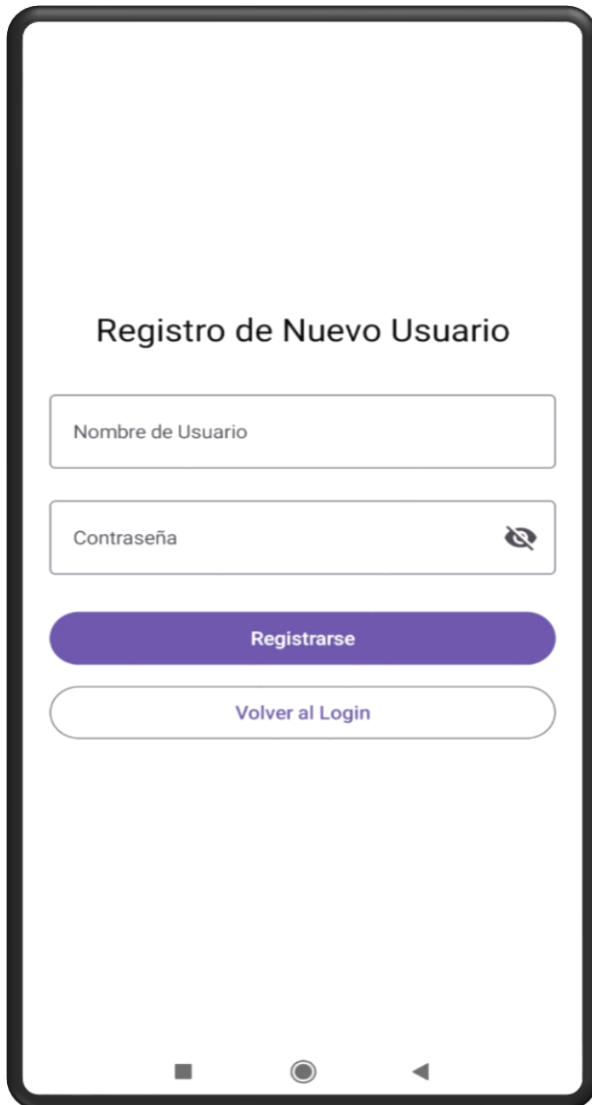
Table: Vehiculos - Items returned (5)

Scan started on June 19, 2025, 23:03:53

☐	placa (String)	activo	anio	color	costoPorDia	imagenResId	marca
☐	ABC123	true	2020	Rojo	50	2130968607	Toyota
☐	TIL777	true	2018	Rojo	35	2130968590	Mazda
☐	AUC455	true	2025	Negro	75	2130968580	Hyundai
☐	DEF456	true	2022	Azul	60	2130968591	Nissan
☐	XYZ789	false	2019	Negro	45	2130968579	Chevrolet

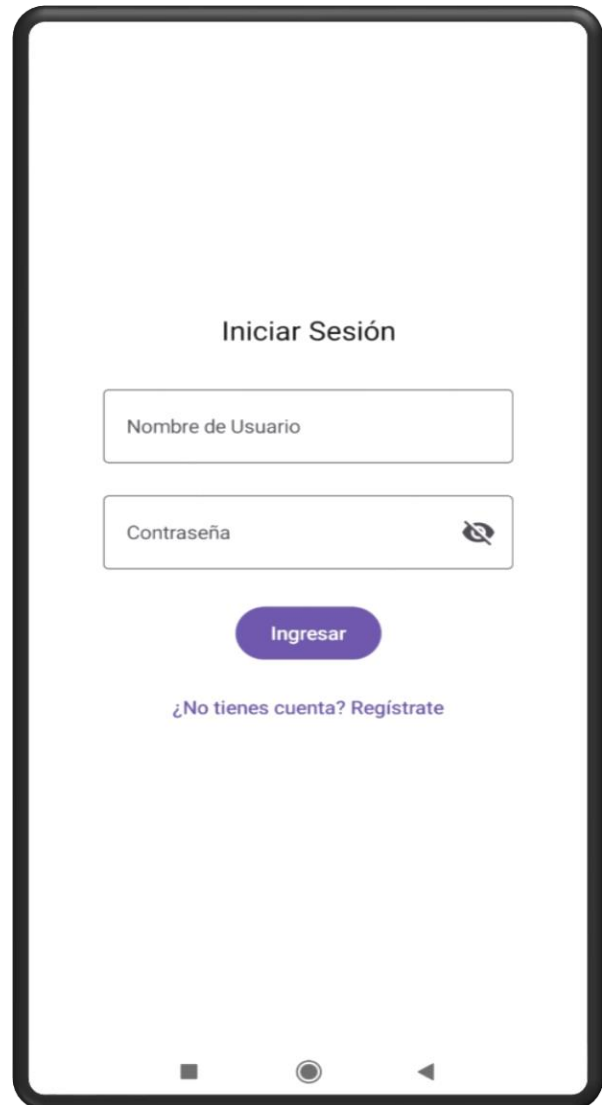
Interfaz de Usuario de la Aplicación

La aplicación cuenta con un sistema básico de autenticación que permite registrar, iniciar sesión y listar usuarios. Esta funcionalidad es esencial para controlar el acceso a otras funciones como la gestión de vehículos.



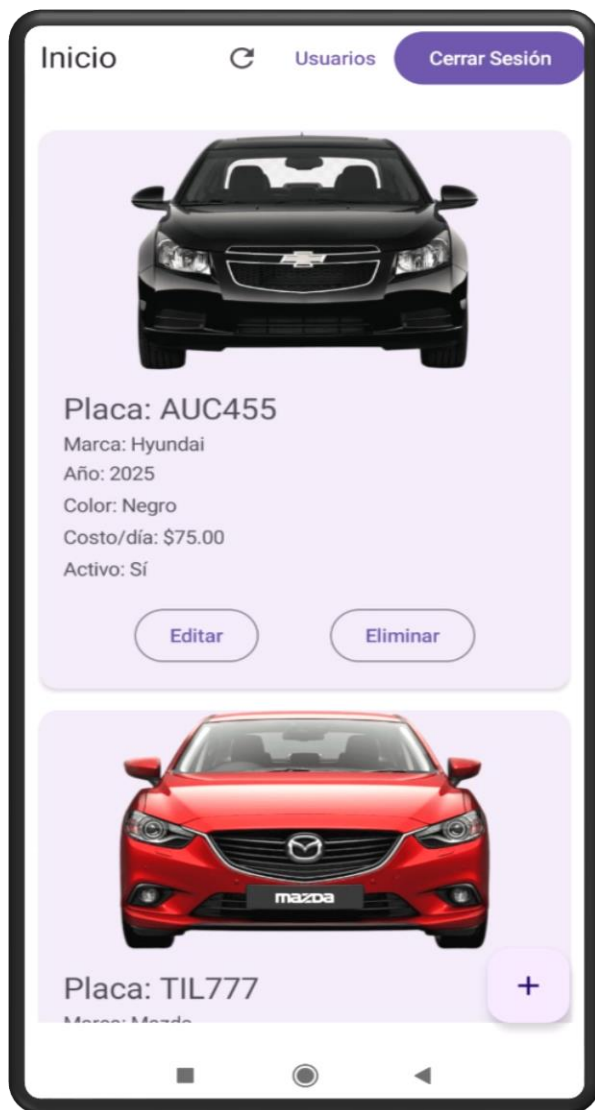
Registro de Usuario

Pantalla que permite crear una nueva cuenta de usuario ingresando un nombre y contraseña. Se incluye una opción para visualizar u ocultar la contraseña. Al presionar el botón “Registrarse”, los datos se almacenan en **Amazon DynamoDB**.



Inicio de Sesión

En esta pantalla, los usuarios ingresan sus credenciales para acceder a las funciones de la aplicación. Se valida el nombre de usuario y contraseña comparando con los registros almacenados en DynamoDB.



Interfaz Principal – Lista de Vehículos

Esta pantalla muestra todos los vehículos registrados en el sistema. Cada tarjeta presenta información como marca, modelo, placa y estado. Los datos se cargan desde **DynamoDB** y se actualizan en tiempo real.



Visualización de Usuarios Registrados

Se presenta una lista de los usuarios registrados en el sistema. Por cada usuario se muestra:

- **Nombre de usuario**
- **Password Hash** (por seguridad, no se muestra la contraseña real)
- **Es Admin:** Indica si el usuario tiene privilegios administrativos

Nuevo Vehículo

Placa

Marca

Año

Color

Costo por día

☒ ¿Activo?

Seleccionar imagen

Guardar

Cancelar

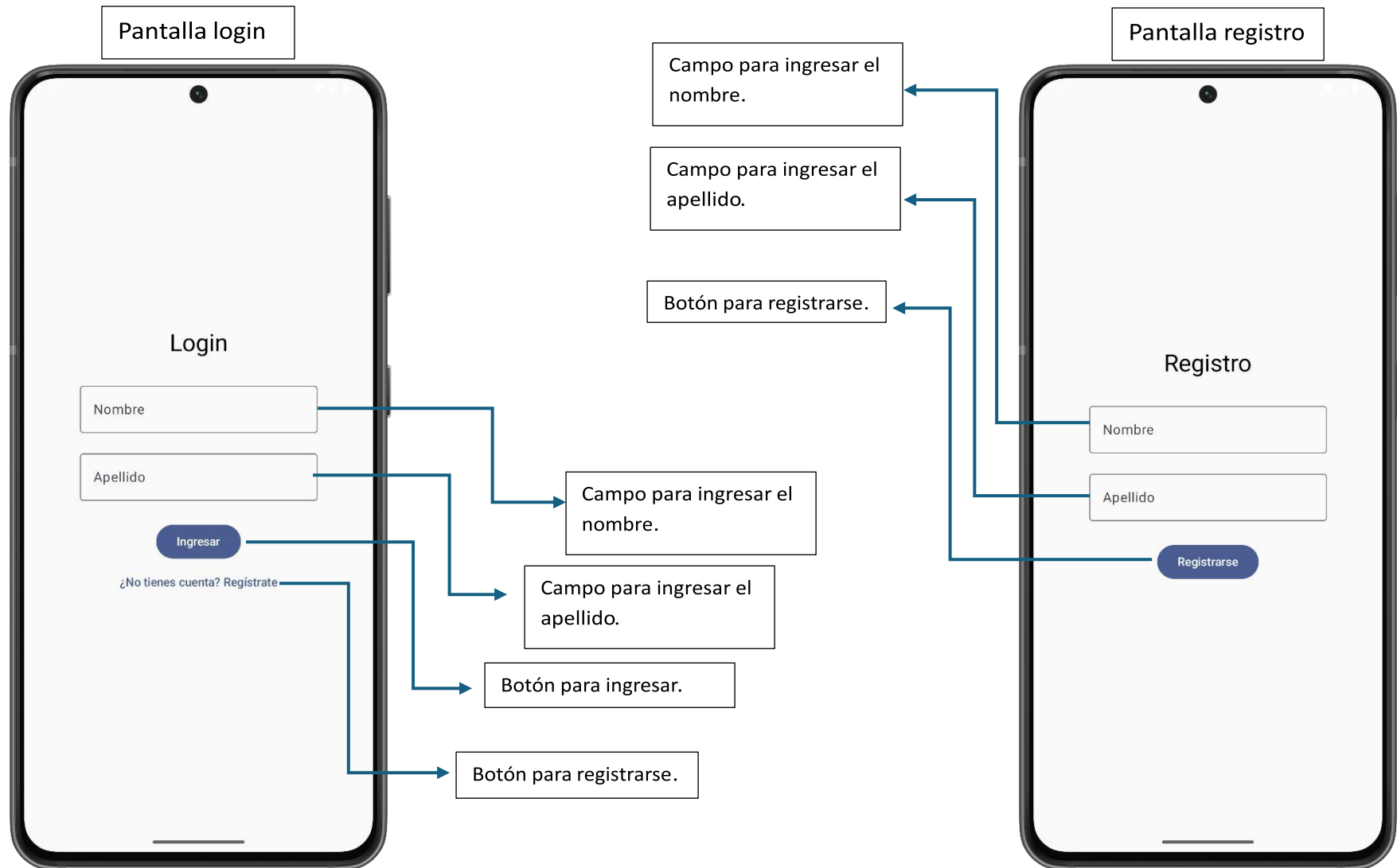
Agregar Vehículo

Pantalla para registrar un nuevo auto ingresando datos como marca, modelo, año, placa y estado.

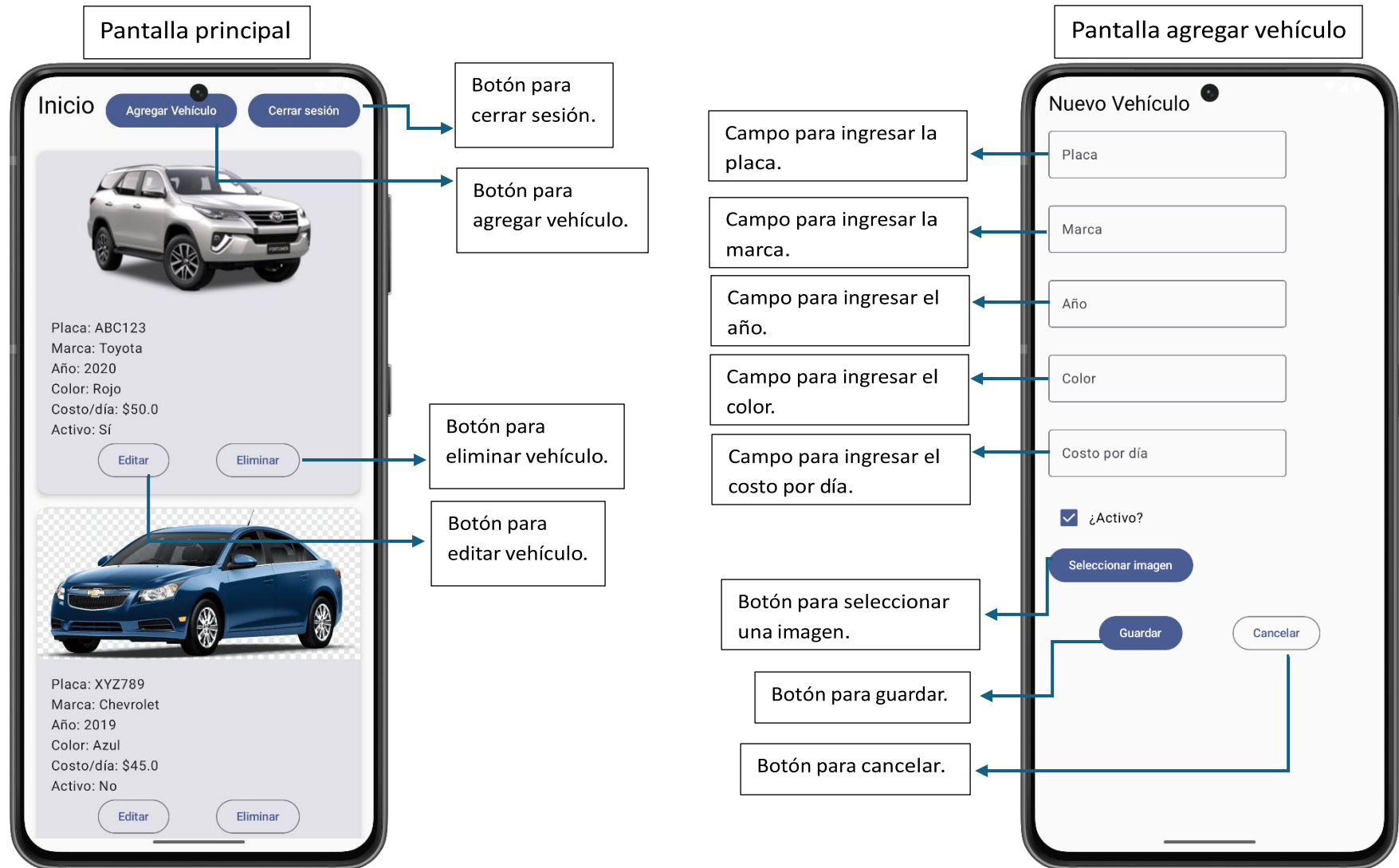
Al presionar Guardar, el vehículo se almacena en **Amazon DynamoDB**.

Se valida que todos los campos estén

Documentación – Mapa de navegación



Documentación – Mapa de navegación



Documentación – Mapa de navegación

